Generative Models and Robust Classifiers

Amirali Malekloo | Guided experimental study | CIFAR-10

In this study, I explored two related questions. First, how do different generative models behave when trained only on CIFAR-10 cats? Second, how does adversarial training affect the response of a classifier to targeted changes in its input? My aim was to understand the behavior of these models through experiments, rather than to propose a new method.

This report is based on the six supplied notebooks and their saved outputs. Training was not rerun for this report. Saved outputs are evidence of earlier executions, but do not guarantee that the current cells reproduce every result without changes. This distinction matters where source code, headings, and stored outputs disagree.

1. Experimental setup

The generative experiments use 32 x 32 CIFAR-10 images and 5,000 cat training examples. StyleGAN1 and StyleGAN2 are unconditional, reduced-size implementations inspired by their papers. BigGan.ipynb also describes an independent BigGAN-style implementation, not the authors' original code. Its saved run selects cat label 3, despite headings and a visible configuration that describe all ten classes. I therefore treat the cat-only output as a historical result, not as proof that the visible default reproduces it.

The SDE notebook embeds pinned upstream score-SDE code and adapts network capacity and training budget. Its 1,000 test cats are used for development monitoring, not gradient updates. Since they inform monitoring, they should not be presented as an untouched final evaluation set.

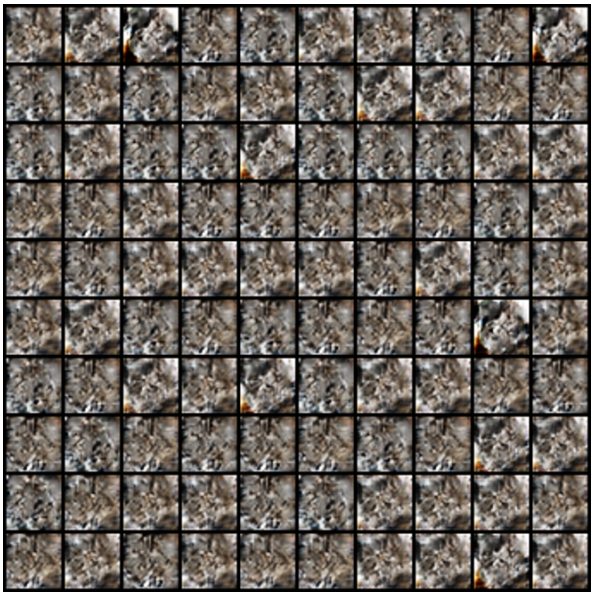
The GAN runs use approximately four-hour session budgets; data downloads consume about an hour in the stored logs. The SDE high preset uses an eight-hour budget. This is not an equal-compute comparison. Optimizer steps are also not directly comparable across these methods.

2. GAN experiments

BigGAN-style training uses a class-conditioned generator, hierarchical latent inputs, attention, a projection discriminator, spectral normalization, hinge losses, EMA, and generator orthogonal regularization. Stored outputs report 2,347,620 generator parameters and 2,375,218 discriminator parameters. The run stops at 66,913 generator steps; total session time is about 225 minutes. Its optional FID cell is skipped, so there is no measured FID result to report.

StyleGAN1 uses an eight-layer mapping network, a learned constant, AdaIN, spatial noise, style mixing, R1 regularization, and EMA. StyleGAN2 instead uses weight modulation/demodulation and path-length regularization. They operate at fixed low resolution and are not full reproductions of the original large-scale benchmarks. The StyleGAN2 output reports 53,501 generator steps and 1,712,032 real-image presentations, about 342.4 passes over 5,000 images.

The original observation was that some GAN samples looked repetitive or blurred, and that more training did not always improve the images. This is a useful motivation to inspect diversity and training stability, but it does not identify the cause. A low discriminator loss or a fluctuating generator loss is not, by itself, evidence of mode collapse. Inactive regularizer fields can be logged as NaN; this alone does not establish numerical failure.



Stored BigGAN-style sample grid; selected from the final truncation outputs. The notebook explores truncation 1.0, 0.8 and 0.5.

3. Style controls and interpretation

Truncation changes the sampling distribution, so samples at different truncation settings should not be treated as the same experiment. StyleGAN samples use psi values 1.0, 0.7 and 0.5. Smaller psi moves styles toward their average. This can change apparent quality and variety without changing the trained weights.

The StyleGAN notebooks also include style mixing, noise-only variation, and interpolation. Noise-only variation keeps the same latent code and changes noise maps; similar images in this test are expected. They should not be used as evidence that independent latent samples have collapsed.



StyleGan1.ipynb: last saved diagnostic figure, included as a diagnostic rather than a random-sample diversity benchmark.

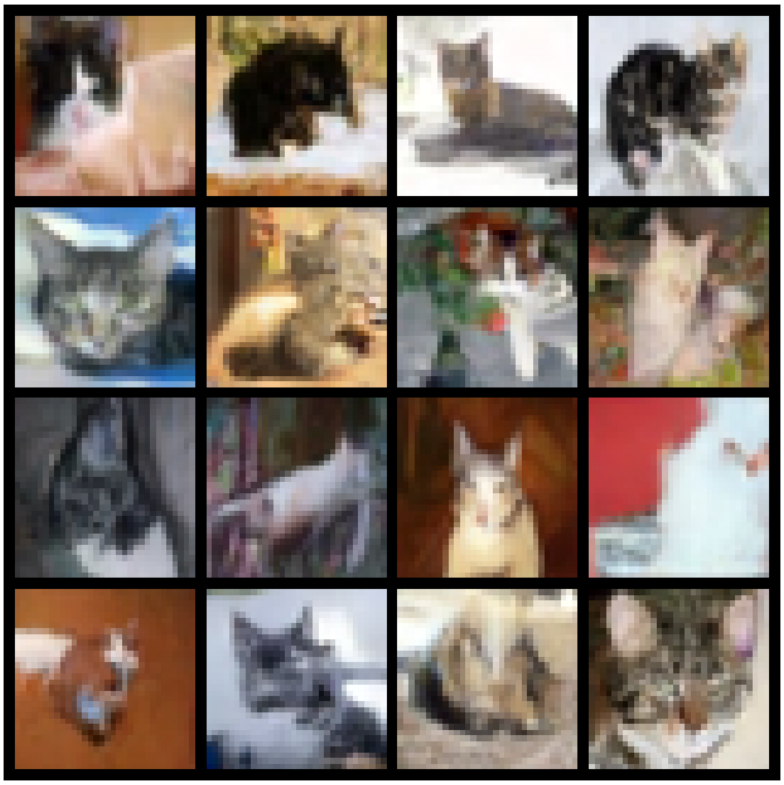


StyleGan2.ipynb: last saved diagnostic figure, included as a diagnostic rather than a random-sample diversity benchmark.

4. Score-SDE experiment

The selected high preset uses a DDPM++ continuous variance-preserving model, width 64, four residual blocks per resolution, and 15,476,675 parameters. Effective batch size is 64, with a measured microbatch of 32. Adam uses a peak learning rate of 0.0002, and the stored warm-up is 400 updates. Training predicts the Gaussian noise added to images at randomly chosen times; generation reverses this corruption using the learned score.

The saved summary reports 94,608 completed updates, 30 skipped updates, and 7.95 hours including setup. Final generation uses 1,000-step Euler-Maruyama sampling with EMA weights. The notebook distinguishes held-out denoising MSE from image-quality metrics. A small noise-prediction loss does not establish good image quality, diversity, or absence of memorization.



Saved final SDE sample grid. These images are direct notebook outputs, not newly generated for the report.

In my experiments, the SDE samples appeared more varied than some GAN outputs. I consider this a qualitative observation about these runs. The longer budget, larger network, different augmentation and different objectives prevent a general conclusion that SDE models outperform GANs.

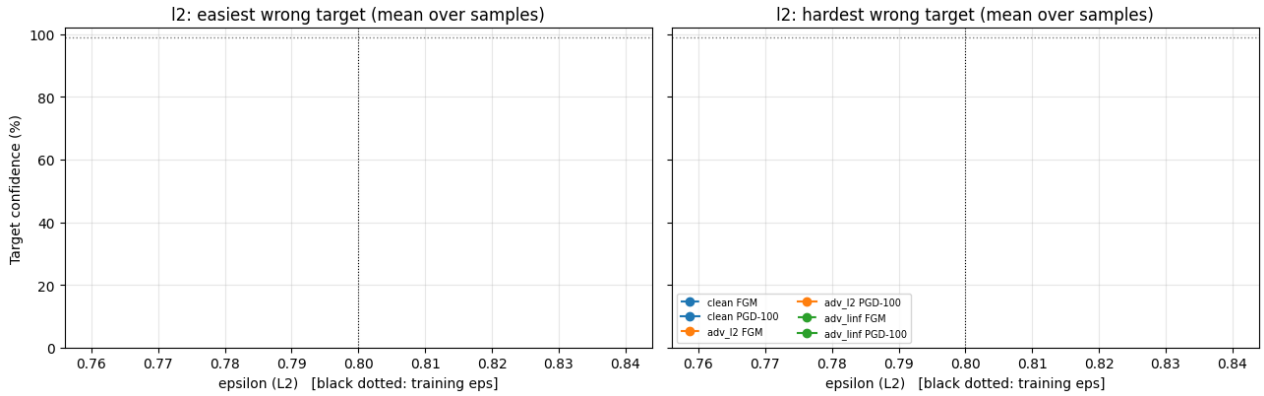
5. Targeted attacks on robust classifiers

ResNet(1).ipynb and PreAct(1).ipynb examine ResNet20 and PreActResNet18 classifiers using clean, L2-adversarially trained, and Linf-adversarially trained checkpoints. This is a separate experiment from training a generative model on cats. At evaluation time, the notebooks load checkpoints and use targeted single-step and iterative attacks to increase a wrong class's probability.

The sweep includes Linf budgets 5, 10, 15, 20 and 25 divided by 255, and L2 budgets 1, 4, 8, 12 and 16. The ResNet saved summary contains eight sample-target pairs. In the current sample-selection code, two random wrong targets are chosen per image. Some markdown and plotting labels still refer to easiest/hardest targets. These inconsistent labels should not be used to claim an easiest-versus-hardest comparison.

For the ResNet20 Linf-trained model, stored mean target confidence under PGD-100 rises from 2.0% at Linf 5/255 to 56.3% at 25/255. Under L2 PGD, it rises from 7.9% at radius 1 to 94.9% at radius 16. The L2-trained model reaches 5.5% at L2 radius 1 and 99.9% at radius 16. These are target-confidence values on selected pairs, not robust accuracy on the test distribution. Large-budget success does not invalidate robustness at a smaller training budget.

Increasing target confidence is also different from producing a realistic image of the target class. The connection to classifier-based image synthesis is a research motivation; these sweeps do not establish a generative-quality benchmark.



Stored ResNet confidence plot. Some category labels are stale relative to the current random-target selection; interpret it with this limitation.

6. What I learned and what remains open

The main result of this study is a set of observations about model behavior. Some GAN configurations showed repetitive outputs, while the SDE run appeared more varied. Robust classifiers resisted small targeted changes more strongly in the recorded examples, but larger perturbations could still produce high target confidence. These observations helped me separate visual quality, diversity, classifier confidence, and robustness: they measure different things.

A stronger comparison would use matched data processing, explicit training and sampling budgets, repeated seeds, fixed sampling settings, and quantitative diversity and memorization checks. For the classifier experiment, a larger independently selected evaluation set and consistent target labels would be necessary. I would also preserve the executed configuration with each saved result, because the visible notebook alone does not always capture the historical run.

Related papers

BigGAN: Large Scale GAN Training for High Fidelity Natural Image Synthesis
https://arxiv.org/abs/1809.11096

StyleGAN: A Style-Based Generator Architecture for Generative Adversarial Networks
https://arxiv.org/abs/1812.04948

StyleGAN2: Analyzing and Improving the Image Quality of StyleGAN
https://arxiv.org/abs/1912.04958

Score-Based Generative Modeling through Stochastic Differential Equations
https://arxiv.org/abs/2011.13456

Image Synthesis with a Single (Robust) Classifier
https://papers.neurips.cc/paper_files/paper/2019/hash/6f2268bd1d3d3ebaabb04d6b5d099425-Abstract.html

Adversarial Examples Are Not Bugs, They Are Features
https://arxiv.org/abs/1905.02175

The first four papers describe the generative-model families. The last two provide background for robust features and classifier-based synthesis; they are related work, not claims of exact reproduction. Original implementation notices, including the embedded score-SDE license and commit, should remain with the code.